QA ROADMAP

From Testing Fundamentals to Automation and SDET Skills

100+ PAGE PRACTICAL MASTER GUIDE

LEARN → PRACTICE → BUILD → DOCUMENT → EXPLAIN → APPLY → INTERVIEW → IMPROVE → GET HIRED → GROW

For freshers, manual testers, automation engineers and aspiring SDETs.

Abhishek Kamal
QA Automation | SDET Career Guidance
Learn. Build. Get Hired.
abhishekkamal.dk+official@gmail.com

How to Use This Roadmap

This is a progression guide, not a syllabus dump. At every stage you should understand the concept, practice it, build something, and be able to explain your decisions.

The four-layer loop

Layer	Question	Evidence
Learn	Do I understand it?	Notes
Practice	Can I perform it?	Exercises
Build	Can I use it realistically?	Project
Explain	Can I defend it?	Interview answer

Core principle

> Do not wait for 100% mastery. Reach the checkpoint, produce evidence, continue, then revisit weak areas.

Master Roadmap

Stage	Destination
01	Career Direction & QA/SDET Mindset
02	Software Testing Fundamentals
03	SDLC, STLC & Agile
04	Requirements & Test Analysis
05	Test Design Techniques
06	Test Cases & Exploratory Testing
07	Defect Management
08	Test Strategy & Risk-Based Testing
09	SQL & Database Testing
10	API & Web Service Testing
11	Programming for QA/SDET
12	Git & Version Control
13	Selenium Automation
14	Playwright Automation
15	PyTest Engineering
16	Automation Framework Architecture
17	Test Data & Configuration
18	API Automation
19	UI + API + DB Integration
20	CI/CD
21	Docker for QA/SDET
22	Performance Testing
23	Security Testing
24	Mobile Testing & Appium
25	Cloud Testing
26	Observability & Production Quality
27	AI/LLM Testing
28	Advanced SDET Engineering
29	Portfolio & GitHub
30	Resume, LinkedIn & Job Search
31	Interview Preparation
32	90-Day Job Roadmap
33	Six-Month SDET Roadmap
34	Career Specializations

Stage	Destination
35	Capstone Project Blueprint
36	Final Checklists & Strategy

01. Career Direction & QA/SDET Mindset

What to learn

• What QA, Automation QA and SDET mean

• Quality ownership and engineering mindset

• Tester vs SDET responsibilities

Why this matters for an SDET

Career Direction & QA/SDET Mindset is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ What QA, Automation QA and SDET mean

■ Quality ownership and engineering mindset

■ Tester vs SDET responsibilities

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

01. Career Direction & QA/SDET Mindset — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Explain QA/SDET in 60 seconds

■ List five product risks

■ Decide what to automate

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

01. Career Direction & QA/SDET Mindset — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving career direction & qa/sdet mindset.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

02. Software Testing Fundamentals

What to learn

• Verification vs validation

• Functional and non-functional testing

• Test levels and test types

Why this matters for an SDET

Software Testing Fundamentals is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Verification vs validation

■ Functional and non-functional testing

■ Test levels and test types

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

02. Software Testing Fundamentals — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Create a testing glossary

■ Build smoke scenarios

■ Classify ten test examples

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

02. Software Testing Fundamentals — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving software testing fundamentals.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

03. SDLC, STLC & Agile

What to learn

• SDLC/STLC

• Scrum and Kanban

• Shift-left testing

Why this matters for an SDET

SDLC, STLC & Agile is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ SDLC/STLC

■ Scrum and Kanban

■ Shift-left testing

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

03. SDLC, STLC & Agile — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Map QA across a sprint

■ Create sprint QA checklist

■ Explain QA before coding

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

03. SDLC, STLC & Agile — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving sdlc, stlc & agile.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

04. Requirements & Test Analysis

What to learn

• Acceptance criteria

• Ambiguity and assumptions

• Traceability

Why this matters for an SDET

Requirements & Test Analysis is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Acceptance criteria

■ Ambiguity and assumptions

■ Traceability

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

04. Requirements & Test Analysis — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Review a login requirement

■ Write clarification questions

■ Build traceability matrix

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

04. Requirements & Test Analysis — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving requirements & test analysis.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

05. Test Design Techniques

What to learn

• Equivalence partitioning

• Boundary value analysis

• Decision tables, states and pairwise

Why this matters for an SDET

Test Design Techniques is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Equivalence partitioning

■ Boundary value analysis

■ Decision tables, states and pairwise

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

05. Test Design Techniques — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Design discount tests

■ Create state model

■ Find boundary cases

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

05. Test Design Techniques — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving test design techniques.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

06. Test Cases & Exploratory Testing

What to learn

• Test case quality

• Checklists

• Exploratory charters

Why this matters for an SDET

Test Cases & Exploratory Testing is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Test case quality

■ Checklists

■ Exploratory charters

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

06. Test Cases & Exploratory Testing — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Write test cases

■ Convert to checklist

■ Run exploratory session

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

06. Test Cases & Exploratory Testing — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving test cases & exploratory testing.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

07. Defect Management

What to learn

• Severity vs priority

• Evidence

• Triage and RCA

Why this matters for an SDET

Defect Management is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Severity vs priority

■ Evidence

■ Triage and RCA

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

07. Defect Management — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Write UI/API/data defects

■ Triage examples

■ Create RCA

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

07. Defect Management — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving defect management.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

08. Test Strategy & Risk-Based Testing

What to learn

• Risk matrix

• Scope and coverage

• Entry/exit criteria

Why this matters for an SDET

Test Strategy & Risk-Based Testing is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Risk matrix

■ Scope and coverage

■ Entry/exit criteria

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

08. Test Strategy & Risk-Based Testing — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Create release strategy

■ Prioritize risks

■ Define release gate

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

08. Test Strategy & Risk-Based Testing — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving test strategy & risk-based testing.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

09. SQL & Database Testing

What to learn

• SELECT, joins, grouping, CTEs

• Data integrity

• Transactions and validation

Why this matters for an SDET

SQL & Database Testing is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ SELECT, joins, grouping, CTEs

■ Data integrity

■ Transactions and validation

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

09. SQL & Database Testing — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Write 25 SQL queries

■ API-to-DB validation

■ Find duplicate/orphan data

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

09. SQL & Database Testing — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving sql & database testing.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

10. API & Web Service Testing

What to learn

• HTTP/REST

• Authentication and authorization

• Schema and negative testing

Why this matters for an SDET

API & Web Service Testing is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ HTTP/REST

■ Authentication and authorization

■ Schema and negative testing

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

10. API & Web Service Testing — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Build API matrix

■ Test error contracts

■ Test boundaries

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

10. API & Web Service Testing — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving api & web service testing.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

11. Programming for QA/SDET

What to learn

• Python or JS/TS

• OOP and reusable code

• Exceptions, modules, async concepts

Why this matters for an SDET

Programming for QA/SDET is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Python or JS/TS

■ OOP and reusable code

■ Exceptions, modules, async concepts

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

11. Programming for QA/SDET — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Solve coding exercises

■ Build API utility

■ Refactor duplication

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

11. Programming for QA/SDET — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving programming for qa/sdet.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

12. Git & Version Control

What to learn

• Repository and commits

• Branches, PRs, merge/rebase

• Conflict resolution

Why this matters for an SDET

Git & Version Control is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Repository and commits

■ Branches, PRs, merge/rebase

■ Conflict resolution

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

12. Git & Version Control — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Create repository

■ Open feature PR

■ Resolve conflict

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

12. Git & Version Control — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving git & version control.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

13. Selenium Automation

What to learn

• WebDriver and locators

• Waits

• POM and Grid awareness

Why this matters for an SDET

Selenium Automation is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ WebDriver and locators

■ Waits

■ POM and Grid awareness

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

13. Selenium Automation — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Automate login/search

■ Create page objects

■ Debug timing failure

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

13. Selenium Automation — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving selenium automation.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

14. Playwright Automation

What to learn

• Locators and auto-waiting

• Contexts and fixtures

• Tracing and network controls

Why this matters for an SDET

Playwright Automation is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Locators and auto-waiting

■ Contexts and fixtures

■ Tracing and network controls

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

14. Playwright Automation — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Build Playwright suite

■ Add API setup

■ Run parallel tests

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

14. Playwright Automation — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving playwright automation.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

15. PyTest Engineering

What to learn

• Fixtures

• Parametrization and markers

• Hooks, reports and parallelism

Why this matters for an SDET

PyTest Engineering is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Fixtures

■ Parametrization and markers

■ Hooks, reports and parallelism

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

15. PyTest Engineering — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Create fixtures

■ Build smoke markers

■ Run selected suite

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

15. PyTest Engineering — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving pytest engineering.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

16. Automation Framework Architecture

What to learn

• Layered architecture

• POM/workflows/API client

• Patterns without over-engineering

Why this matters for an SDET

Automation Framework Architecture is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Layered architecture

■ POM/workflows/API client

■ Patterns without over-engineering

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

16. Automation Framework Architecture — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Draw architecture

■ Refactor scripts

■ Create review checklist

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

16. Automation Framework Architecture — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving automation framework architecture.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

17. Test Data & Configuration

What to learn

• Factories and builders

• Environment configuration

• Secrets and isolation

Why this matters for an SDET

Test Data & Configuration is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Factories and builders

■ Environment configuration

■ Secrets and isolation

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

17. Test Data & Configuration — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Build data factory

■ Create config strategy

■ Design cleanup

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

17. Test Data & Configuration — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving test data & configuration.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

18. API Automation

What to learn

• Reusable API client

• Schema/contract validation

• Chaining and correlation

Why this matters for an SDET

API Automation is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Reusable API client

■ Schema/contract validation

■ Chaining and correlation

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

18. API Automation — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Automate endpoints

■ Add schemas

■ API-to-DB assertions

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

18. API Automation — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving api automation.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

19. UI + API + DB Integration

What to learn

• Hybrid automation

• Backend state validation

• Eventual consistency

Why this matters for an SDET

UI + API + DB Integration is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Hybrid automation

■ Backend state validation

■ Eventual consistency

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

19. UI + API + DB Integration — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ End-to-end checkout

■ API data setup

■ DB state check

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

19. UI + API + DB Integration — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving ui + api + db integration.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

20. CI/CD

What to learn

• Pipeline stages

• Jenkins/GitHub Actions

• Artifacts and quality gates

Why this matters for an SDET

CI/CD is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Pipeline stages

■ Jenkins/GitHub Actions

■ Artifacts and quality gates

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

20. CI/CD — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Build pipeline

■ Publish reports

■ Add smoke gate

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

20. CI/CD — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving ci/cd.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

21. Docker for QA/SDET

What to learn

• Images and containers

• Dockerfile/Compose

• Reproducible execution

Why this matters for an SDET

Docker for QA/SDET is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Images and containers

■ Dockerfile/Compose

■ Reproducible execution

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

21. Docker for QA/SDET — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Dockerize tests

■ Run dependencies

■ Debug container failure

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

21. Docker for QA/SDET — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving docker for qa/sdet.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

22. Performance Testing

What to learn

• Load/stress/spike/endurance

• Workload modelling

• p95/p99 and bottlenecks

Why this matters for an SDET

Performance Testing is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Load/stress/spike/endurance

■ Workload modelling

■ p95/p99 and bottlenecks

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

22. Performance Testing — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Model checkout traffic

■ Run baseline

■ Explain regression

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

22. Performance Testing — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving performance testing.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

23. Security Testing

What to learn

• Auth/RBAC

• OWASP awareness

• Security regression

Why this matters for an SDET

Security Testing is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Auth/RBAC

■ OWASP awareness

■ Security regression

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

23. Security Testing — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Build authorization matrix

■ Test object access

■ Check headers/secrets

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

23. Security Testing — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving security testing.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

24. Mobile Testing & Appium

What to learn

• Native/hybrid/mobile web

• Device matrix

• Permissions/network/lifecycle

Why this matters for an SDET

Mobile Testing & Appium is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Native/hybrid/mobile web

■ Device matrix

■ Permissions/network/lifecycle

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

24. Mobile Testing & Appium — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Build Appium suite

■ Test offline

■ Create device matrix

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

24. Mobile Testing & Appium — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving mobile testing & appium.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

25. Cloud Testing

What to learn

• Compute/storage/database

• IAM/networking

• Logs and metrics

Why this matters for an SDET

Cloud Testing is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Compute/storage/database

■ IAM/networking

■ Logs and metrics

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

25. Cloud Testing — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Map cloud architecture

■ Cloud test checklist

■ Investigate logs

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

25. Cloud Testing — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving cloud testing.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

26. Observability & Production Quality

What to learn

• Logs, metrics, traces

• SLI/SLO/SLA

• Production smoke and incident evidence

Why this matters for an SDET

Observability & Production Quality is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Logs, metrics, traces

■ SLI/SLO/SLA

■ Production smoke and incident evidence

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

26. Observability & Production Quality — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Build dashboard concept

■ Synthetic API check

■ Write incident timeline

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

26. Observability & Production Quality — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving observability & production quality.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

27. AI/LLM Testing

What to learn

• Golden datasets

• RAG evaluation

• Safety and prompt testing

Why this matters for an SDET

AI/LLM Testing is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Golden datasets

■ RAG evaluation

■ Safety and prompt testing

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

27. AI/LLM Testing — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Create evaluation cases

■ Build rubric

■ Compare versions

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

27. AI/LLM Testing — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving ai/llm testing.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

28. Advanced SDET Engineering

What to learn

• Reliability

• Flaky-test engineering

• Architecture and quality gates

Why this matters for an SDET

Advanced SDET Engineering is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Reliability

■ Flaky-test engineering

■ Architecture and quality gates

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

28. Advanced SDET Engineering — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Create retry policy

■ Classify failures

■ Review framework

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

28. Advanced SDET Engineering — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving advanced sdet engineering.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

29. Portfolio & GitHub

What to learn

• Flagship project

• README and evidence

• Architecture documentation

Why this matters for an SDET

Portfolio & GitHub is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Flagship project

■ README and evidence

■ Architecture documentation

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

29. Portfolio & GitHub — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Build ShopKart

■ Publish evidence

■ Document decisions

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

29. Portfolio & GitHub — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving portfolio & github.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

30. Resume, LinkedIn & Job Search

What to learn

• ATS positioning

• Profile proof

• Application pipeline

Why this matters for an SDET

Resume, LinkedIn & Job Search is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ ATS positioning

■ Profile proof

■ Application pipeline

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

30. Resume, LinkedIn & Job Search — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Tailor resume

■ Improve profile

■ Track applications

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

30. Resume, LinkedIn & Job Search — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving resume, linkedin & job search.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

31. Interview Preparation

What to learn

• Testing/SQL/API

• Coding/automation

• Project and senior scenarios

Why this matters for an SDET

Interview Preparation is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Testing/SQL/API

■ Coding/automation

■ Project and senior scenarios

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

31. Interview Preparation — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Mock interviews

■ Gap tracker

■ Project deep dive

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

31. Interview Preparation — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving interview preparation.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

32. 90-Day Job Roadmap

What to learn

• Foundation

• Build

• Apply

Why this matters for an SDET

90-Day Job Roadmap is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Foundation

■ Build

■ Apply

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

32. 90-Day Job Roadmap — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ 15/30/60/90 day gates

■ Weekly scorecard

■ Job-ready assessment

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

32. 90-Day Job Roadmap — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving 90-day job roadmap.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

33. Six-Month SDET Roadmap

What to learn

• Month-by-month progression

• Specialization

• Evidence

Why this matters for an SDET

Six-Month SDET Roadmap is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Month-by-month progression

■ Specialization

■ Evidence

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

33. Six-Month SDET Roadmap — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Set monthly outcomes

■ Review skills

■ Choose branch

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

33. Six-Month SDET Roadmap — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving six-month sdet roadmap.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

34. Career Specializations

What to learn

• Performance

• Security

• Cloud

• AI/LLM

• Mobile

• SRE

Why this matters for an SDET

Career Specializations is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Performance

■ Security

■ Cloud

■ AI/LLM

■ Mobile

■ SRE

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

34. Career Specializations — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Compare paths

■ Choose one

■ Build specialization proof

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

34. Career Specializations — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving career specializations.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

35. Capstone Project Blueprint

What to learn

• Requirements to release

• UI/API/DB

• CI/CD/performance/security

Why this matters for an SDET

Capstone Project Blueprint is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Requirements to release

■ UI/API/DB

■ CI/CD/performance/security

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

35. Capstone Project Blueprint — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Create architecture

■ Build test strategy

■ Produce final report

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

35. Capstone Project Blueprint — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving capstone project blueprint.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

36. Final Checklists & Strategy

What to learn

• Readiness gates

• Career loop

• Long-term growth

Why this matters for an SDET

Final Checklists & Strategy is not valuable because it is a keyword. It is valuable when it helps you understand risk, create reliable evidence, automate useful work, or make better engineering decisions.

Core checklist

■ Readiness gates

■ Career loop

■ Long-term growth

Readiness questions

• Can I explain the concept without notes?

• Can I give a realistic product example?

• Can I identify a common failure?

• Can I explain what I would automate and why?

• Can I collect useful evidence when it fails?

36. Final Checklists & Strategy — Practical Lab

Use this lab as a portfolio-oriented exercise. Keep assumptions explicit and document your result.

Execution flow

1. Choose a realistic feature or system.
2. Define business risk and expected behavior.
3. Create positive, negative and boundary scenarios.
4. Execute once manually to understand the behavior.
5. Automate repeatable checks where automation provides value.
6. Capture evidence such as logs, screenshots, traces, reports or database output.
7. Write a short conclusion: what passed, what failed, what remains risky.

Checkpoint tasks

■ Self-assess

■ Plan next 30 days

■ Commit to evidence

Definition of done

■ Another person can reproduce the exercise.

■ The README/setup is understandable.

■ At least one negative or boundary case exists.

■ Failures produce diagnostic evidence.

■ The final result is documented.

PORTFOLIO RULE: Every major learning stage should create at least one visible artifact.

36. Final Checklists & Strategy — Interview & Review Sheet

Rapid revision

Question	Your answer
What is it?	
Why does QA need it?	
Real example?	
Common mistake?	
How would you debug it?	
What would you automate?	
What trade-off exists?	

Scenario drill

You join a product team with incomplete documentation and an urgent release involving final checklists & strategy.

• What questions do you ask first?

• What risks do you prioritize?

• What would you test first?

• What evidence would you collect?

• How would you communicate the result?

Self-rating

Skill	1	2	3	4	5
Concept	■	■	■	■	■
Hands-on	■	■	■	■	■
Debugging	■	■	■	■	■
Interview	■	■	■	■	■

The QA/SDET Skill Dependency Map

Testing → SQL/API → Programming → UI automation → Framework → CI/CD → Advanced quality → Portfolio → Interviews.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

The Job-Ready Gate

Apply seriously when you can explain testing, SQL, APIs, programming, automation, framework architecture and one project deeply.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

The Automation Decision Tree

Repeated + stable + valuable + diagnosable = strong automation candidate. Exploratory/judgment-heavy/changing behavior may need human testing.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

The Debugging Loop

Failure → reproduce → collect evidence → isolate layer → identify root cause → fix → rerun → prevent recurrence.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

The Release Gate

Smoke → critical regression → defects → environment → integrations → performance/security checks → monitoring/rollback → release.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

The Interview Recovery Loop

Rejection → identify exact gap → practice targeted gap → mock → reassess → reapply.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

The Career Growth Loop

Skill → evidence → ownership → impact → communication → larger scope → specialization/leadership.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

The SDET Weekly Operating System

> Learn 25% → practice 30% → build 25% → interview 10% → career/profile 10%.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

The Flagship Project

ShopKart: requirements, manual strategy, API, DB, UI, framework, CI/CD, Docker, performance, security, observability.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

Portfolio Evidence Checklist

Architecture diagram, risk matrix, test strategy, code, pipeline, reports, defects, README, performance result, security checklist.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

Resume Formula

Action + technology + scope + measurable result + business/quality impact.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

Interview Formula

Concept → example → implementation → failure → debugging → trade-off → project evidence.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

First 90 Days

Days 1–15 foundations; 16–30 programming/API/Git; 31–50 UI automation; 51–65 framework; 66–75 CI/CD; 76–90 profile/applications/interviews.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

Six-Month Progression

Month 1 fundamentals; 2 programming/API; 3 automation; 4 framework/DB; 5 CI/CD/Docker/advanced; 6 portfolio/interviews/specialization.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

Specialization Decision

Choose performance, security, cloud, mobile, AI/LLM, SRE/reliability or platform engineering based on interest + market + evidence.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

What Recruiters Need to See

Clear positioning, credible experience, relevant stack, project evidence, concise impact and communication.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

What Senior SDETs Do Differently

They reason about risk, reliability, maintainability, environments, observability, failure classification and trade-offs.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

Common Fresher Mistakes

Tool hopping, tutorial dependency, no project, no GitHub evidence, memorized answers, avoiding coding, waiting for perfect readiness.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

Common Automation Mistakes

Blind automation, sleeps everywhere, giant page objects, global state, uncontrolled retries, duplicated data, weak diagnostics.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

Final Strategy

Learn less randomly, build more deliberately, document evidence, explain decisions, apply consistently and improve from feedback.

Action worksheet

Question	Write your answer
What will I do?	
What will I build?	
What evidence will I produce?	
What could go wrong?	
When will I review it?	

Printable Worksheet 1: Weekly Learning Planner

Item	Notes / Decision	Status
1		■
2		■
3		■
4		■
5		■
6		■
7		■
8		■
9		■
10		■
11		■
12		■

Use this page repeatedly. Keep the latest version in your study/project notes.

Printable Worksheet 2: Risk Matrix

Item	Notes / Decision	Status
1		■
2		■
3		■
4		■
5		■
6		■
7		■
8		■
9		■
10		■
11		■
12		■

Use this page repeatedly. Keep the latest version in your study/project notes.

Printable Worksheet 3: Test Design Sheet

Item	Notes / Decision	Status
1		■
2		■
3		■
4		■
5		■
6		■
7		■
8		■
9		■
10		■
11		■
12		■

Use this page repeatedly. Keep the latest version in your study/project notes.

Printable Worksheet 4: Defect Investigation Sheet

Item	Notes / Decision	Status
1		■
2		■
3		■
4		■
5		■
6		■
7		■
8		■
9		■
10		■
11		■
12		■

Use this page repeatedly. Keep the latest version in your study/project notes.

Printable Worksheet 5: Automation Review

Item	Notes / Decision	Status
1		■
2		■
3		■
4		■
5		■
6		■
7		■
8		■
9		■
10		■
11		■
12		■

Use this page repeatedly. Keep the latest version in your study/project notes.

Printable Worksheet 6: Framework Review

Item	Notes / Decision	Status
1		■
2		■
3		■
4		■
5		■
6		■
7		■
8		■
9		■
10		■
11		■
12		■

Use this page repeatedly. Keep the latest version in your study/project notes.

Printable Worksheet 7: Interview Gap Tracker

Item	Notes / Decision	Status
1		■
2		■
3		■
4		■
5		■
6		■
7		■
8		■
9		■
10		■
11		■
12		■

Use this page repeatedly. Keep the latest version in your study/project notes.

Printable Worksheet 8: Project Tracker

Item	Notes / Decision	Status
1		■
2		■
3		■
4		■
5		■
6		■
7		■
8		■
9		■
10		■
11		■
12		■

Use this page repeatedly. Keep the latest version in your study/project notes.

Printable Worksheet 9: Job Application Tracker

Item	Notes / Decision	Status
1		■
2		■
3		■
4		■
5		■
6		■
7		■
8		■
9		■
10		■
11		■
12		■

Use this page repeatedly. Keep the latest version in your study/project notes.

Printable Worksheet 10: 30-Day Action Plan

Item	Notes / Decision	Status
1		■
2		■
3		■
4		■
5		■
6		■
7		■
8		■
9		■
10		■
11		■
12		■

Use this page repeatedly. Keep the latest version in your study/project notes.

Printable Worksheet 11: 60-Day Action Plan

Item	Notes / Decision	Status
1		■
2		■
3		■
4		■
5		■
6		■
7		■
8		■
9		■
10		■
11		■
12		■

Use this page repeatedly. Keep the latest version in your study/project notes.

Printable Worksheet 12: 90-Day Action Plan

Item	Notes / Decision	Status
1		■
2		■
3		■
4		■
5		■
6		■
7		■
8		■
9		■
10		■
11		■
12		■

Use this page repeatedly. Keep the latest version in your study/project notes.

Final QA/SDET Career Strategy

CHOOSE → LEARN → PRACTICE → BUILD → PROVE → POSITION → APPLY → INTERVIEW → ANALYZE → IMPROVE → GET HIRED → OWN PROBLEMS → GROW

The objective is not to memorize the largest QA syllabus. The objective is to become an engineer who can understand a product, identify risk, write useful code, automate intelligently, investigate failures, communicate clearly and improve software quality.

Your next action

Pick the earliest roadmap stage where you cannot confidently demonstrate the checkpoint. Start there. Build evidence. Then move forward.

Abhishek Kamal — QA Automation | SDET Career Guidance

Learn. Build. Get Hired.

abhishekkamal.dk+official@gmail.com